CS & IT ENGINEERING



Algorithms

Dynamic Programming



Lecture No.- 05

By- Aditya sir

Recap of Previous Lecture



Topic

Topic

0/1 Knapsack

Floyd Warshall

LCS

Topics to be Covered



- Topic
- Topic
- Topic

LCS
MCM
} v. imp



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Dynamic Programming

Longest Common Sub-Sequence (LCS):

- Based on Strings
- A sequence/group/array of one or more characters.



Topic : Dynamic Programming

String:

1) Substring:

- A contiguous sequence of 1 or more characters from the given string.



Topic : Dynamic Programming

String:

2) Subsequence:

- A sequence of 1 or more characters from the given string, that may or may not be contiguous, but their relative order is maintained.



Topic : Dynamic Programming

Example:

- String: "ABC" → length = 3
- Sub-strings: A, B, C → 3

  AB, BC → 2

  ABC → 1

No. of substrings = 3 + 2 +1 = 6



Topic : Dynamic Programming

Important Result:

Given : S[1, ..., n]

#Q. What are the total number of sub-strings of at-least length 1 (non-empty sub-strings)?



Topic : Dynamic Programming

Sub-Sequence:

- String = "ABC"
- Sub-Sequence = A, B, C, AB, BC, AC, ABC → Valid sub-sequence
- CA, CB → relative order not maintained, not a sub-sequence.



Topic : Dynamic Programming

Important:

- Every substring is also a sub-sequence, but a Sub-sequence may or may not be a substring.



[MSQ]

#Q. Given a string S = "CABDABBCD".

Which of the following are valid sub-sequences?

A BBD

B BAC

C CDCA

D CADC

D ADABD



Topic : Dynamic Programming

Longest Common Sub-Sequence:

#Q. Given a string of length 'n' characters, the number of sub-sequences possible are?

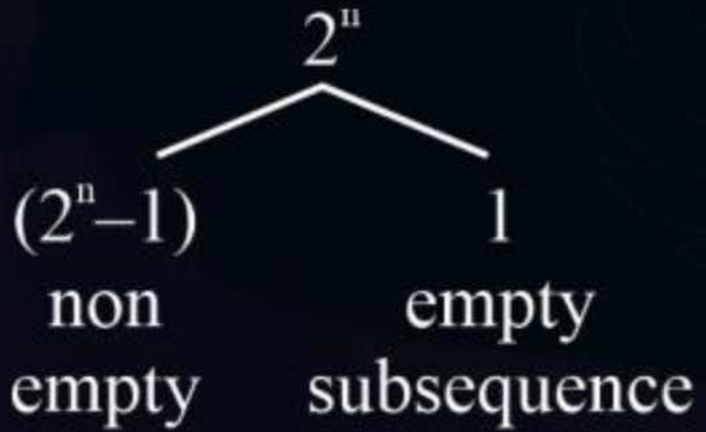


Topic : Dynamic Programming

Longest Common Sub-Sequence:



$\phi \rightarrow$ empty sub-sequence





Topic : Dynamic Programming

Brute Force/Enumeration:

All Subsequence → A

All Subsequence → B

- $O(2^n)$
- Hence, we need DP based approach.



Topic : Dynamic Programming

Longest Common Subsequence (DP based approach):

Common Subsequence:

- Given two strings X and Y of length n and m characters, a subsequence that is common to both X and Y is known as a common sub-sequence.



Topic : Dynamic Programming

Example:

#Q. X = "ABCD", Y = "BDC"

Which sub-sequences are common to both?

1) A
2) B
3) BC
4) BD
5) BCD
6) DC
7) BDC



Topic : Dynamic Programming

LCS Problem Statement:

#Q. Given strings 'A' and 'B' of length (Characters) 'n' & 'M' respectively. The LCS problem is to determine the subsequence that is of the longest length among all the subsequences that are common to both 'A' & 'B'?



Topic : Dynamic Programming

Example:

S1 = "ABCBDAB"

S2 = "BDCABA"

Longest LCS = 4



Topic : Dynamic Programming

Common Subsequences:

AB → 2

BCA → 3

BCB → 3

BCBA → 4

BCAB → 4

- Multiple longest common subsequences are possible.
- But LCS length is unique → 4



Topic : Dynamic Programming

DP based solution to LCS:

- Let 'i' and 'j' be indices in the strings 'X' and 'Y' respectively, such that.

$X = [x_1, x_2, x_3, \ldots, x_n]$

$Y = [y_1, y_2, y_3, \ldots, y_m]$



Topic : Dynamic Programming

- Let L(i, j) represent the length of the common sub-sequence of the strings X and Y.

$$L(i, j) = 1 + L(i-1, j-1); \text{ if } X[i] == Y[j]$$

$$= \max\{L(i-1, j), L(i, j-1)\}; \text{ if } X[i] \neq Y[j]$$

$X = x_1 x_2 \cdots x_n$ (i)

$Y = y_1 y_2 \cdots y_m$ (j)



Topic : Dynamic Programming

Initialization: (For 0 based indexing)

- L[-1, j] = 0,

  1st string is exhausted

  if i = –1; j = –1, 0, 1, 2, …, m–1

- L[i, –1] = 0,

  2nd string is exhausted

  if j = –1; i = –1, 0, 1, 2, …, n–1

$X = x_0 x_1 \cdots x_{n-1}$

$Y = y_0 y_1 \cdots y_{m-1}$

X = " "

Y = "abcd…"



Topic : Dynamic Programming

Example:

Case1: X[i] = Y[j]

X = "ABBCDCCABA"

Y = "CBDADCBDAA"

1 + L(i-1, j-1)



Topic : Dynamic Programming

Example:

Case2: $X[i] \neq Y[j]$

X = "ABCDBBCCADB"

Y = "BDBCAAABDCD"

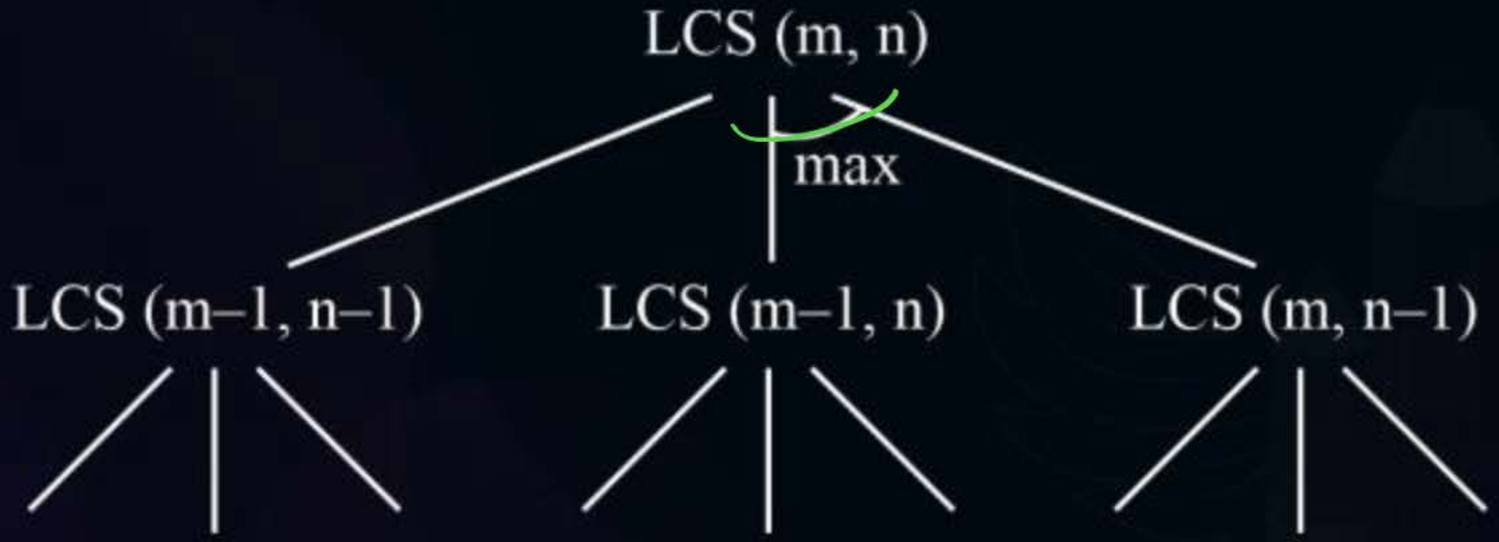
$\max\{L(i, j-1), L(i-1, j)\}$



Topic : Dynamic Programming

Overlapping Subproblems:

$|X| = n, |Y| = m$

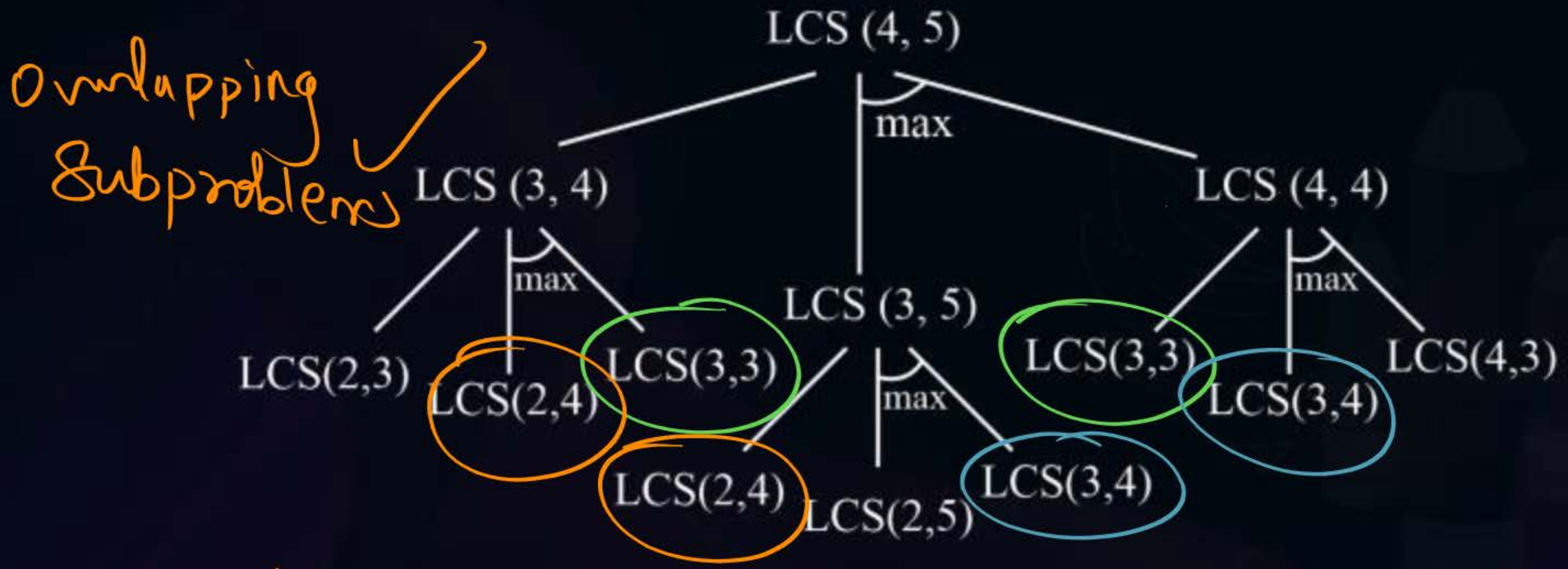




Topic : Dynamic Programming

Overlapping Subproblems:

$|X| = 4, |Y| = 5$





Topic : Dynamic Programming

Example:

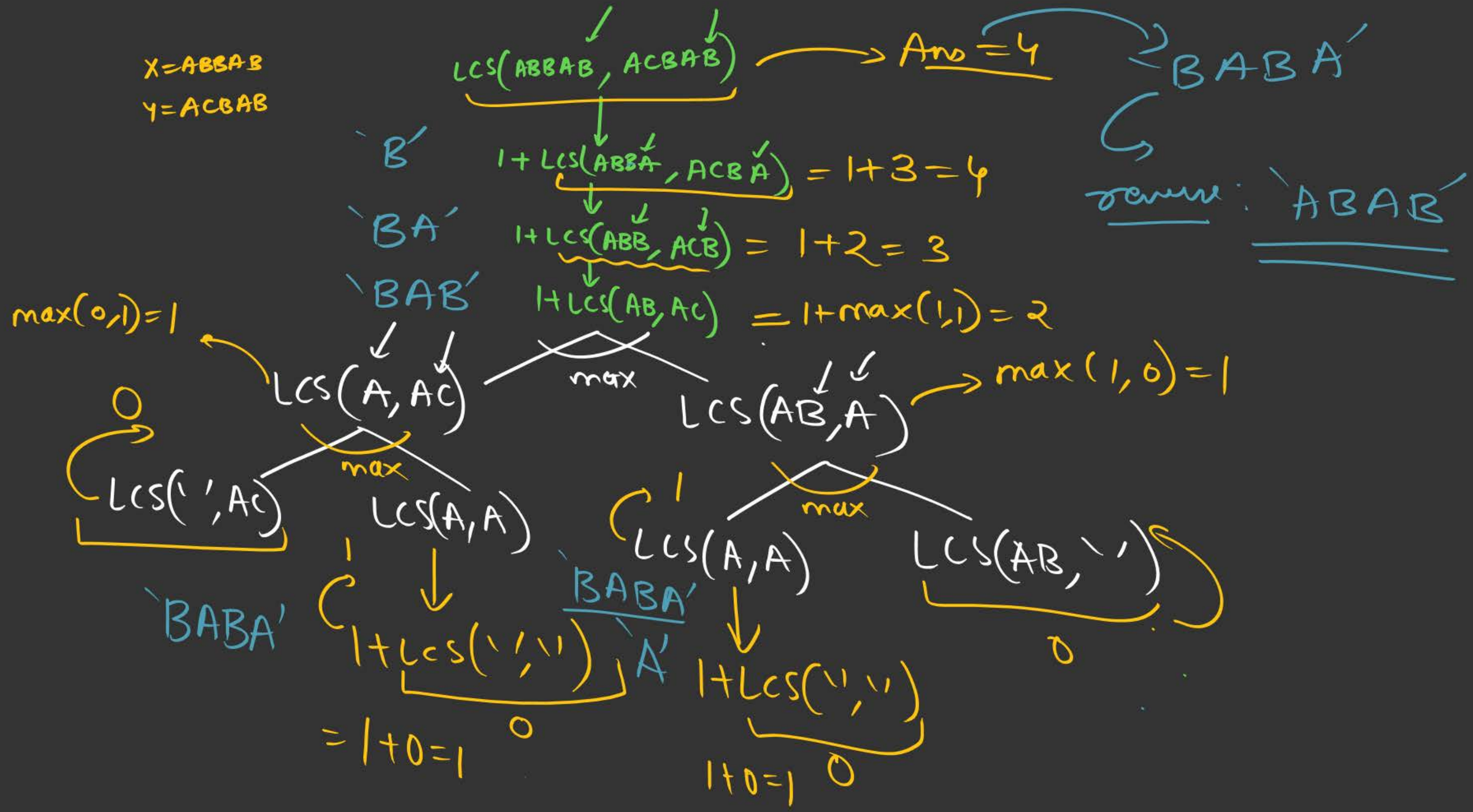
Top-Down (Memoization/Recurrence):

X = "ABBAB"

Y = "ACBAB"

LCS : "ABAB"

Length = 4


X=ABBAB
Y=ACBAB
LCS(ABBAB, ACBAB)
Ans = 4
BABA
reverse: ABAB
'B'
1+LCS(ABBA, ACBA) = 1+3 = 4
'BA'
1+LCS(ABB, ACB) = 1+2 = 3
'BAB'
1+LCS(AB, AC) = 1+max(1,1) = 2
max(0,1)=1
max
LCS(A, AC)
LCS(AB, A)
max(1,0)=1
0
max
LCS(' ', AC)
LCS(A, A)
1
max
LCS(A, A)
LCS(AB, ' ')
'BABA'
BABA
'A'
1+LCS(' ', ' ')
0
=1+0=1
1+LCS(' ', ' ')
0
1+0=1
0



Topic : Dynamic Programming

Example:

Bottom-up Approach (Tabulation):

X = "ABCBDAB" → length = 7

Y = "BDCABA" → length = 6

Topic : Dynamic Programming



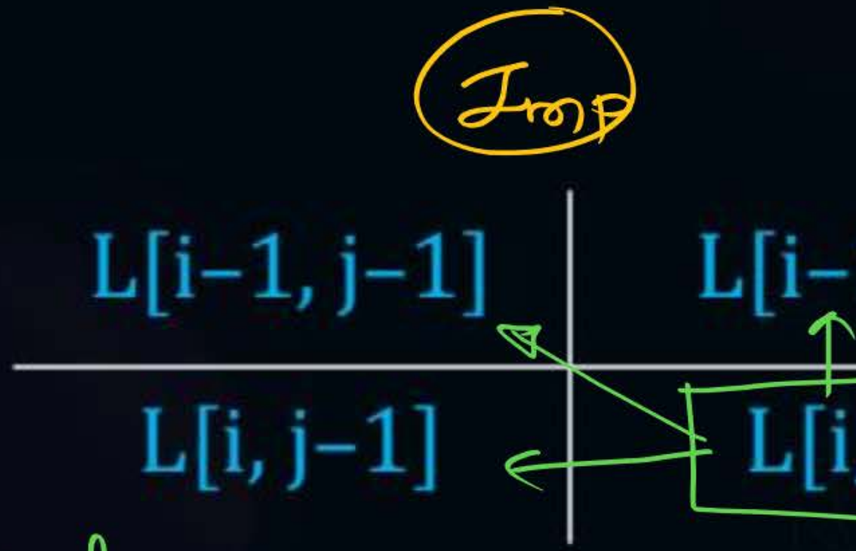
X[N+1, M+1]

		0	1	2	3	4	5	6
		" "	B	D	C	A	B	A
0	" "	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

final ans



Topic : Dynamic Programming



$X_i = Y_j$ = 1 + diagonal

$X_i \neq Y_j$ = max(Left, Up)

"BDAB"

↳ (4)



Topic : Dynamic Programming

LCS Recurrence:

- $LCS(i, j) = 1 + LCS(i-1, j-1)$; if $X[i] = Y[j]$
- $LCS(i, j) = \max\{L(i, j-1), L(i-1, j)\}$; if $X[i] \neq Y[j]$



Topic : Dynamic Programming

Algorithm LCS based on Bottom-up Tabulation method :

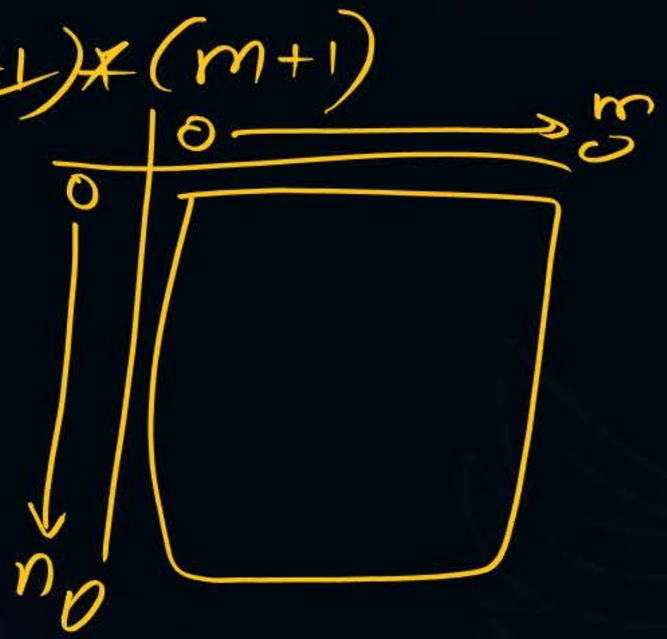
Algorithm LCS (x, y)

Integer x[0..n], y [0..m];

{

integer L [0..n , 0..m];

1. for i ← 0 to n
   L [i, 0] = 0;
2. for j ← 0 to m
   L [0 , j] = 0;
3. for i ← 1 to n
   for j ← 1 to m





Topic : Dynamic Programming

```
if (x [i] = y [j]) then
        L[I, j] = 1 + L [I – 1, j – 1];
else
        L[i, j] = max {L [i, j – 1], L [i – 1, j]};
```

Complexity Analysis :-

TC : O(n x m)

SC : O(n x m)


MCM



Topic : Dynamic Programming

Matrix Chain Multiplication/Product (MCM):

#Q. Given two matrices A and B of size m×n and n×p.

How many scalar multiplication are reqruied in $A_{m\times n} * B_{n\times p} = C$?



Topic : Dynamic Programming

Case 1:

Square Matrices:

$$A_{n\times n} * B_{n\times n} = C_{n\times n}$$

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

2×2 2×2 2×2



Topic : Dynamic Programming

$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21}$ — (2)

$c_{12} = a_{11} \times b_{12} + a_{12} \times b_{22}$ — (2)

$c_{21} = a_{21} \times b_{11} + a_{22} \times b_{21}$ — (2)

$c_{22} = a_{21} \times b_{12} + a_{22} \times b_{22}$ — (2)

Every element = 2

Hence, total scalar multiplication = $2 \times (2 \times 2) = 2 \times 2 \times 2$ = 8

{ Sq mat: $n \times n$ } $\Rightarrow n^3$

A, B



Topic : Dynamic Programming

Case 2:

Non-Square Matrices:



$$A_{m\times n} * B_{n\times p} = C_{m\times p}$$

- Hence, for all $m\times p$ element.

Total scalar multiplication = $n\times(m\times p) = m \times n \times p$



Topic : Dynamic Programming

Important Result:

$$A_{m\times n} * B_{n\times p} = C_{m\times p}$$

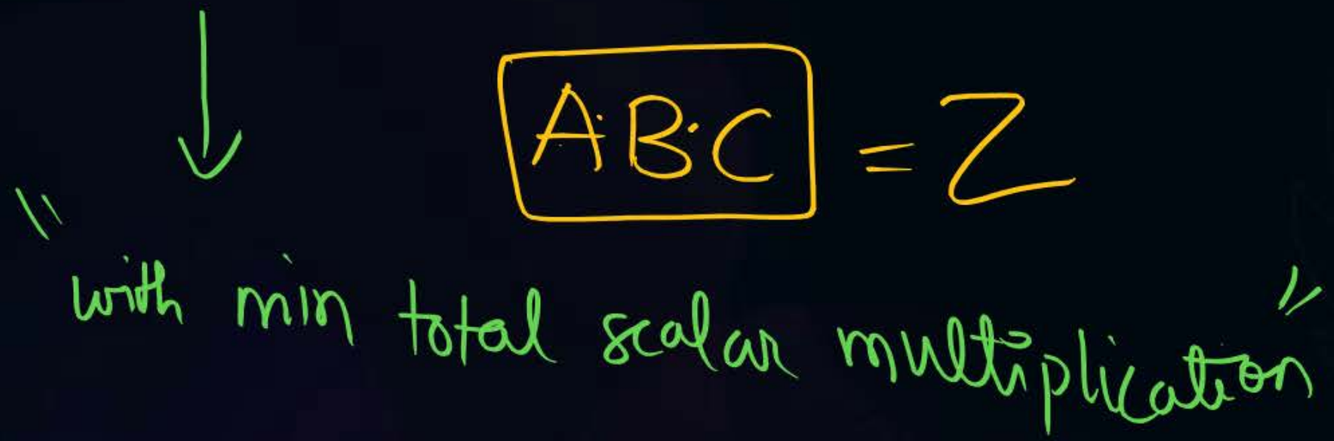
- Total number of scalar multiplication = $m \times n \times p$

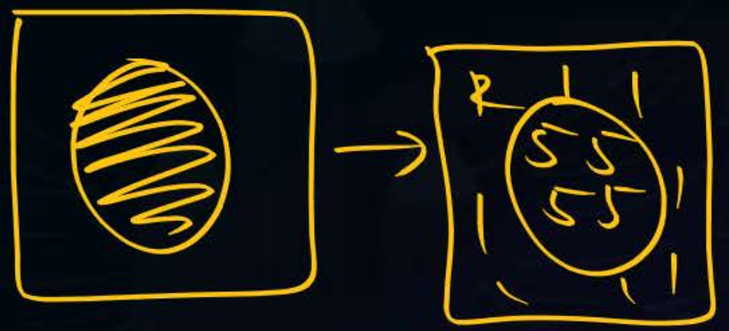


Topic : Dynamic Programming

MCM Problem Statement:

- Given a chain of compatable non-square matrices, it is required to multiply them together to get a final resultant matrix.







Topic : Dynamic Programming

Example:

n = 3

$[A_{2\times10}, B_{10\times50}, C_{50\times20}] = (ABC) = Z_{2\times20}$

A(BC)



Topic : Dynamic Programming

Parenthesizing Problem:

ABC = (1) (A * B) * C ⇒ 3000

(2) A * (B * C) ⇒ 10,400

A → 2x10

B → 10x50

C → 50x20

2x10 10x50

2x10x50

① A(BC)

② (AB)C

2x50



Topic : Dynamic Programming

(1) (A * B) * C

How many Scalar multiplication?

(i) (A * B) = 2 × 10 × 50

(ii) (A * B) * C = 2 × 50 × 20

So, total scalar multiplication for (A * B) * C are

= 2 × 10 × 50 + 2 × 50 × 20

= 1000 + 2000 = 3000



Topic : Dynamic Programming

(2) A * (B * C)

(i) (B * C) = 10 × 50 × 20

(ii) A * (B * C) = 2 × 10 × 20

So, total scalar multiplication for A * (B * C) are

= 10 × 50 × 20 + 2 × 10 × 20

= 1000 + 400

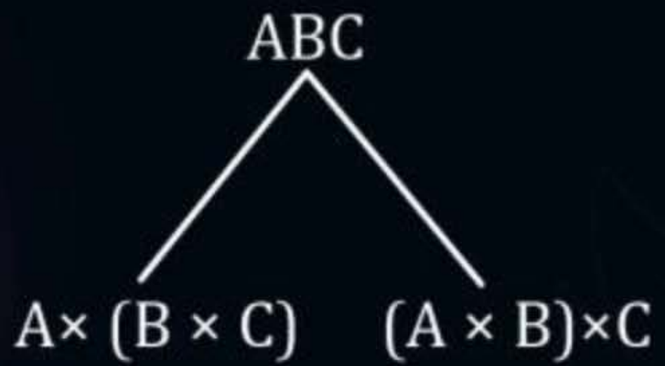
= 10400



Topic : Dynamic Programming

MCM objective function

to minimize the total number of scaler multiplication.





Topic : Dynamic Programming

DP based approach:-

$A_1 \rightarrow A_n$

- Given a chain of n matrixes (A1, A2 .... An) where matrix Ai is of Dimension $P_{i-1} * P_i$.

  The MCM problem is to fully parenthesize the chain such that the total number of scalar multiplication are minimized.

A B C

1 2 3

P_0P_1 P_1P_2 P_2P_3

$P = [10, 2, 20, 30]$

A C

B

Topic : Dynamic Programming

V. imp

K: 1 → n-1

#Q. How many ways to multiply chain n matrices?

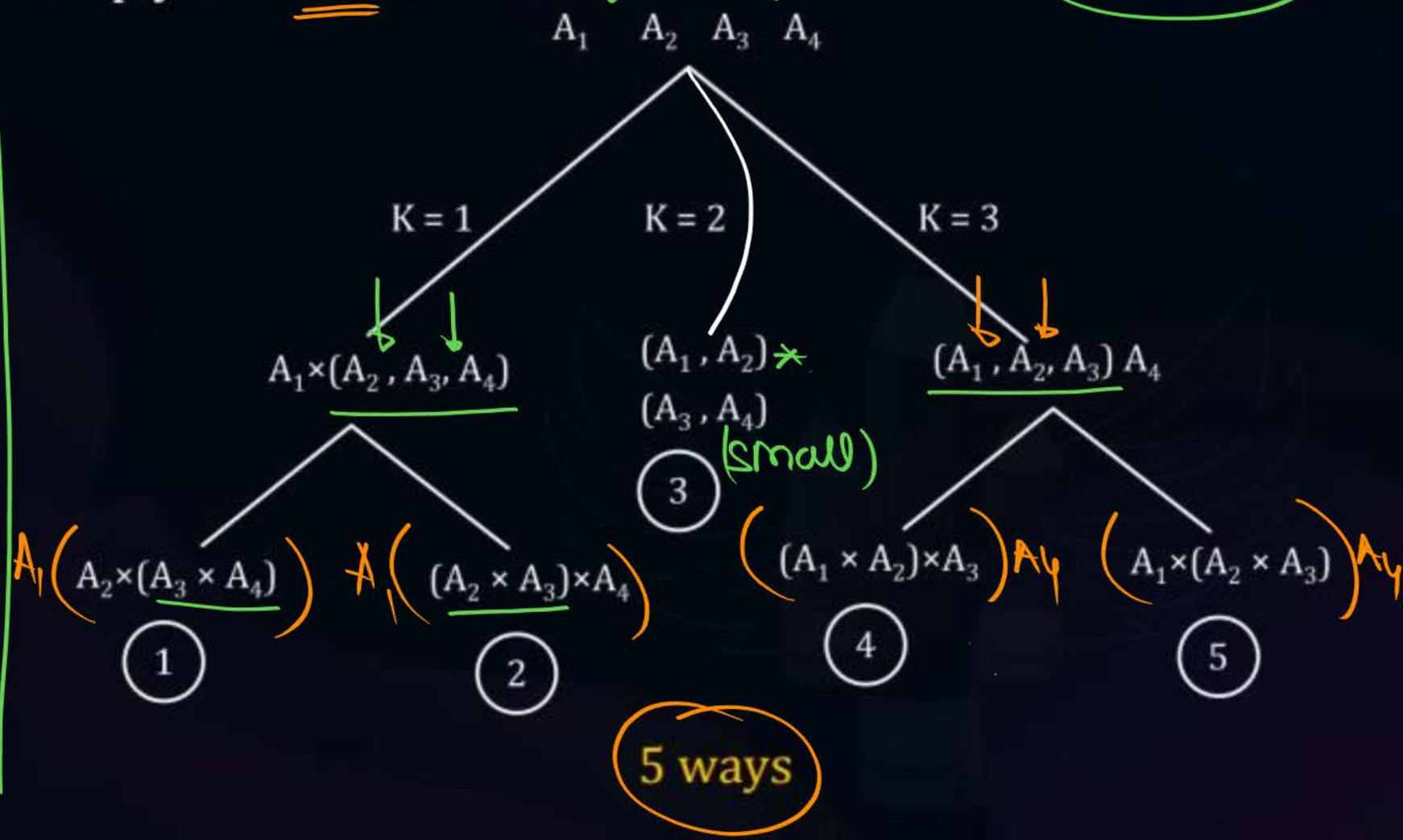
K = 1 K=2

ABC

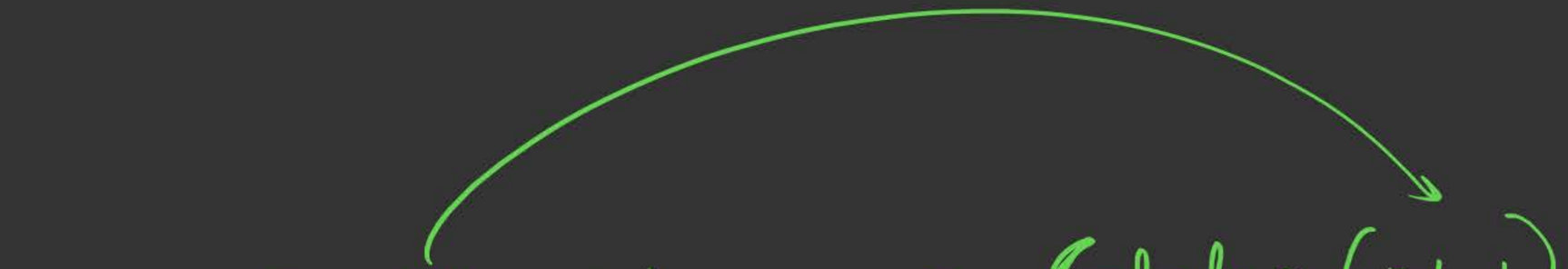
(1) A*(B*C)

(2) ~~(B*C)*C~~ (A*B)*C

2 ways



5 ways



N+1 matrices ⟹ Catalan (N)

eg:- 3 matrices

↓

$$\text{Catalan}(2) = \frac{1}{(2+1)} * {}^{4}C_{2} = \frac{1}{3} \times \frac{4!}{2!\,2!} = \frac{1}{3} \times \frac{4 \times 3}{2} = 2$$



Topic : Dynamic Programming

Generalized Results:-

Total no. of ways to multiply a chain of (n + 1) matrix ⇒ Catalan number (n)

E.g. N = 3

↓

4 matrix

↓↓

Catalan (3)

$$= \frac{1}{4} \times {}^{6}C_3$$

$$= \frac{1}{4} \times \frac{6 \times 5 \times 4 \times 3}{3! \times 3!}$$

$$= \frac{6 \times 5}{3} = 5$$

$$\frac{1}{(n+1)} \times {}^{2n}C_n$$

n + 1 → n

n ⇒ n-1



Topic : Dynamic Programming

n Matrix $\rightarrow \frac{1}{n} \times^{2(n-1)} C_{(n-1)}$

Catalan (N-1)



Topic : Dynamic Programming

Deamination of DP based Recurrence for men:

Let the resultant matrix Ai j be of product

$(A_i * Ai+1 * A_{i+2} \ldots\ldots A_j)$

Chain → Result Ai j

$A_1 \times A_2 \times A_3 \ldots\ldots A_n \Rightarrow A_{1\times n}$

k = (1 to n–1)



Topic : Dynamic Programming

Any optimal Parameterization must split the chain about the matrix A_k & A_{k+1} ~~rule~~ Such that that the number of scaler multiplications are minimum.

$(A_1 \cdot A_k \; A_{k+1} A_n)$

MCM → Bottom Up



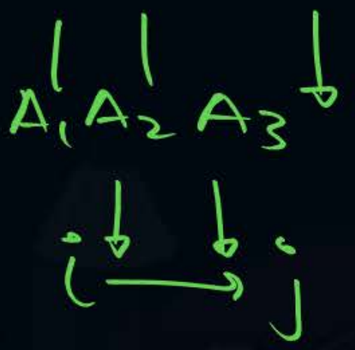
Topic : Dynamic Programming

DP Recurrence

$m[i,j] \Rightarrow A_{ij}$

Let m[i, j] represent the number of scalar multiplication to get the resultant matrix Ai j.

$[A_i * A_{i+1} \ldots\ldots\ldots A_k * \ldots\ldots A_j] \Rightarrow A\ i\ j \rightarrow M\ [i, j]$



Split at k

$(A_i * A_i * \ldots\ldots A_k) \times (A_{k+1} * \ldots\ldots\ldots A_j)$ Where $\rightarrow$ $i \le k \le (j-1)$

A_{ik} $A_{k+1,j}$

m [i , k] m [k+1, j]



Topic : Dynamic Programming

$$M[i, j] = \min \{m[i,k] + m[k+1, j] + P_{i-1} * P_k * P_j\}$$

For all $k \rightarrow i$ to $j-1$

$m[i,k]$ → $(A_i \ldots\ldots A_k) * (A_{k+1} \ldots\ldots A_j)$ ← $m[k+1, j]$

$P_{i-1}*P_i$ $\quad$ $P_{k-1}*P_k$ $\quad$ P_k*P_{k+1} $\quad$ $P_{j-1}*P_j$

$P_{i-1}*P_k$ $\quad$ P_k*P_j

$R * R$

$P_{i-1}*P_k$ $\quad$ $P_k * P_j$

$(P_{i-1} * P_k * P_j)$



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

LCS ✓

MCM ≈ 70 %.



THANK - YOU